

Tutorial 2: Newton's Method

Newton's method is an algorithm for approximating the roots of a function. For instance, to find an approximation of $\sqrt{a}$ (with $a > 0$) we can use Newton's method to find a root of $f(x) = x^2 - a$. That is, we want to find the value of x (the “root”) such that $f(x) = x^2 - a = 0$. In this case, the algorithm is:

1. Start with a guess x_0 for the root.
2. Repeat until the approximation is “good enough”: $x_{i+1} = \frac{1}{2} \left(x_i + \frac{a}{x_i} \right)$.

So if $a = 2$ and our initial guess is $x_0 = 2$, we have:

$$x_1 = \frac{x_0}{2} + \frac{1}{x_0} = 1 + \frac{1}{2} = \frac{3}{2} = 1.5$$
$$x_2 = \frac{x_1}{2} + \frac{1}{x_1} = \frac{3}{4} + \frac{2}{3} = \frac{17}{12} = 1.416666\dots$$

So after two steps, we already have an answer that is good to two decimal places, since $\sqrt{2} = 1.41421\dots$

Exercise 1: Edit the `newton.c` file to implement the following code to find the square root of a positive number a :

- Write a function `double sqrt_iter(double x, double a)` that takes x and a , and returns $\frac{1}{2} \left(x + \frac{a}{x} \right)$.
- Starting from the values provided in `newton.c`, use a loop to call `sqrt_iter` ten times. Update guess and print the approximation after each iteration.

Exercise 2: stop when accurate enough.

The first program always performs ten iterations. Instead, use a stopping condition so that the number of iterations depends on how quickly the approximation improves.

Create a new file called `newton2.c`. Copy your `sqrt_iter` function into it, then:

- set `a = 2`, `guess = 1` and `epsilon = 0.000001`;
- use a `while` loop to continue as long as

$$\text{guess}^2 < a - \text{epsilon} \quad \text{or} \quad \text{guess}^2 > a + \text{epsilon};$$

Hint: note that in C you can implement guess^2 by doing `guess * guess`.

- count how many Newton iterations are performed;
- print the final approximation and the iteration count.

Test your program with $a = 2$, $a = 9$ and $a = 100$, always using a positive initial guess. Why is a `while` loop more suitable than a `for` loop here? What would happen if the initial guess were zero?